Design Document

1. Problem statement

Supply-chain operations teams need evidence from prior disruption cases when assessing a new event. The application retrieves comparable records and summarizes impact, response, and recovery outcomes.

2. Business objective

Support initial triage and evidence review through a local interface. The user supplies event facts and receives retrieved case evidence, optional model estimates, response comparisons, and an explicit human-review signal.

3. Solution overview

The existing backend combines structured MySQL records, Chroma semantic retrieval, BM25 term retrieval, four saved scikit-learn models, and a LangGraph workflow. FastAPI exposes the functionality to the existing React dashboard and the added Streamlit client. LLM explanation generation is optional and defaults to demo mode.

4. Architecture design

Dataset inspection and ingestion run as offline scripts. Separate scripts build the vector and keyword indexes and train models. At runtime, FastAPI loads persisted artifacts and runs the workflow. The workflow retrieves candidate records, enriches them from MySQL, computes analyses, and validates the recommendation against retrieved response types and evidence count. See architecture/Architecture_Diagram.pdf.

5. Component explanation

- backend/app/main.py: API routes, startup artifact loading, persistence, and response assembly.
- backend/app/rag/: case-to-text conversion and hybrid Chroma/BM25 retrieval.
- backend/app/agents/workflow.py: LangGraph nodes for classification, investigation, retrieval, prediction, response, recovery, resilience, priority, recommendation, and validation.
- backend/app/ml/: shared feature definitions, loading, inference, and feature importance for saved pipelines.
- backend/app/services/analysis.py: outcome summaries, priority percentiles, recommendation scoring, and resilience heuristic.
- scripts/: explicit dataset, index, and model build steps.
- frontend/ and streamlit_app.py: two local clients of the same HTTP API.

6. Technology selection

FastAPI provides request validation and generated OpenAPI documentation. SQLAlchemy and MySQL hold structured records and workflow history. ChromaDB stores vectors with cosine distance, while BM25 provides lexical ranking. Sentence Transformers creates normalized BAAI/bge-small-en-v1.5 embeddings. scikit-learn supplies persisted tabular estimators. LangGraph organizes the sequential workflow as named nodes. Streamlit provides a small Python client suitable for a local demonstration; React/Vite remains the existing richer interface. demo is the default LLM setting so local analysis does not require a key.

7. Data pipeline

The CSV inspector profiles schema, types, missing values, duplicates, categories, ranges, and IQR outliers. The ingestion script trims text, converts columns, parses booleans, removes malformed rows and duplicate IDs, checks bounds, and inserts unseen IDs into MySQL in batches. Index and model builds are explicit scripts; API startup never rebuilds them.

8. Retrieval strategy

The query encoder produces a normalized embedding. Chroma returns cosine-distance candidates, converted to a bounded similarity score 1 - distance / 2. BM25 scores tokenized query terms against tokenized documents. Both rank a candidate pool, and SQL enforces requested metadata filters. The final score normalizes BM25 by the pool maximum and combines semantic and keyword scores using configurable weights (defaults 0.7 and 0.3), then returns the top K. No-result retrieval returns an empty list.

9. Chunking strategy

The index uses one complete rendered record per disruption, with no sliding token windows or overlap. This preserves all case metadata and outcomes together. Each record stores its disruption_id and categorical metadata. Since the source is tabular and the rendered records are short, per-record documents avoid splitting a case across chunks. The code does not define a token-size or overlap parameter.

10. Embedding approach

The configurable Sentence Transformer defaults to BAAI/bge-small-en-v1.5; the vector builder requests normalized vectors. Chroma persists the vectors at backend/data/chroma in a collection configured by CHROMA_COLLECTION, with cosine as the collection distance. The code does not declare an embedding dimension constant; it is supplied by the selected model.

11. Agent design

One LangGraph workflow contains specialized nodes, not separate autonomous agents. It classifies severity, constructs retrieval filters, searches cases, predicts four outcomes, summarizes strategy and recovery results, calculates a project-specific resilience heuristic and priority score, selects a supported response, optionally asks an LLM to produce JSON, and validates evidence coverage. Historical recommendations require at least three cases; low coverage or invalid validation sets the human-review flag.

12. API architecture

FastAPI validates JSON with Pydantic models and returns JSON. SQLAlchemy dependency injection manages request sessions for database-backed handlers. CORS origins come from ALLOWED_ORIGINS. Full routes, inputs, outputs, and error codes are in docs/API_Documentation.pdf.

13. Evaluation strategy

Model training compares Random Forest and Extra Trees on a validation split and records held-out test metrics. Current saved metrics cover 100,000 rows split into 70,000 train, 15,000 validation, and 15,000 test records. DeepEval can judge saved analyses for faithfulness, answer relevancy, contextual relevancy, contextual precision, recommendation quality, and explanation quality when a provider is configured. No LLM-judge results are claimed in the current report.

14. Security

Secrets use local environment variables loaded from .env; .gitignore excludes .env, keys, PEMs, credentials, and secret files. API keys are never hard-coded. The event description is length-limited and rejects several instruction-like patterns. The LLM prompt constrains recommendations to response types observed in retrieved cases and labels uncertainty. Authentication is not implemented, so the service is intended for local use.

15. Error handling

Pydantic returns HTTP 422 for invalid requests. Missing records and recommendations return 404. Missing retrieval or prediction artifacts return 503. LLM transport, configuration, or output failures fall back to deterministic historical recommendation logic. The health route reports database and artifact availability. Streamlit shows API error details and connection failures.

16. Challenges

The offline build sequence must be run before runtime retrieval succeeds. The vector and keyword indexes must represent the same number of cases. Training can take time and local model downloads require network access or a pre-cached model. The project stores derived model and index files outside version control.

17. Trade-offs

Separate build scripts make startup predictable but require setup steps. Hybrid retrieval combines semantic and term-level evidence but uses fixed configurable weights rather than a learned ranker. Saved tabular models are easy to run locally but their validity depends on the supplied dataset. The default no-LLM path improves reproducibility and avoids key setup, while generated narrative is optional.

18. Limitations

Dataset provenance is not recorded and the data contains no event date or supplier identity. Historical associations are not causal estimates. Recommendation quality has no measured LLM-judge score in the checked-in evidence. Recovery and revenue models show material prediction error in the held-out metrics. The resilience value is a project heuristic, not a validated standard. Authentication and production deployment are outside the local project scope.

19. Future improvements

Record and verify data provenance, add an explicit timestamp and supplier entity only if a suitable source supports them, evaluate retrieval against labeled query-case pairs, calibrate the supplier-change classifier, add drift and subgroup analysis, and persist a reproducible evaluation set with human-reviewed labels.